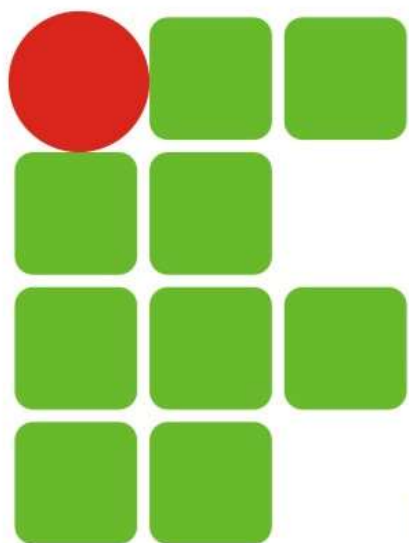




**INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA**
RIO GRANDE DO NORTE

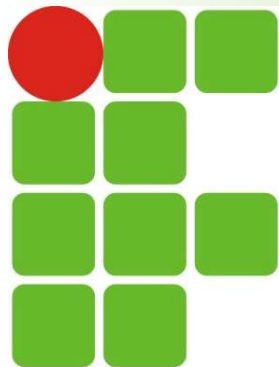


Curso de Tecnologia em Redes de Computadores

Disciplina: Introdução aos Sistemas Abertos

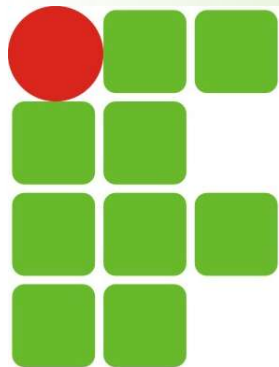
02. Noções básicas de uso do Shell

Prof. Ronaldo <ronaldo.maia@escolar.ifrn.edu.br>



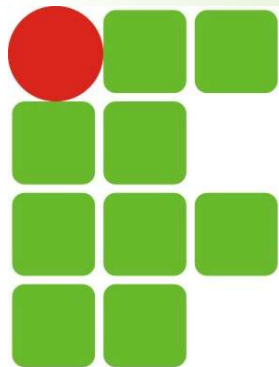
Antes de começar...

- Para melhor fixação do conteúdo, é fundamental que comandos e exemplos apresentados no material e/ou em sala sejam executados
 - Através da máquina virtual Ubuntu Server 22.04.4
 - https://drive.google.com/file/d/1MoUC_WBVbj0KkHhrT3ty-Ng_Im_BH4i3/view?usp=sharing
 - Após conclusão de seu download, no **VirtualBox**, clique no menu "*Arquivo*" => "*Importar Appliance*" e selecione o arquivo *ubuntu-22.04.4-server-amd64.ova*
 - Informações da máquina Linux:
 - **Usuário:** isa - **Senha:** isa
 - Ou utilizando um terminal online (caso não consiga executar a VM)
 - <https://bellard.org/jslinux/vm.html?url=alpine-x86.cfg&mem=192&authuser=0>
 - <https://copy.sh/v86/?profile=linux26&authuser=0>



Interpretador de comandos

- Também conhecido como "shell". É o programa responsável por interpretar as instruções enviadas pelo usuário e seus programas ao S.O (o kernel);
- O Shell executa comandos lidos do dispositivo de entrada padrão (teclado) ou de um arquivo executável;
- O Shell é a principal ligação entre o usuário, os programas e o kernel;
- O Linux possui diversos interpretadores de comandos, entre os mais comuns estão o bash, sh e o ksh.
 - O bash é o mais utilizado nas distribuições Linux



Interpretador de comandos

- Aviso de comando (prompt)
 - É a linha mostrada na tela para digitação de comandos
 - A posição onde o comando será digitado é marcado por um traço ou quadro piscante chamado "cursor"
 - O aviso de comando do usuário root é identificado pelo símbolo **#**, e o aviso de comando de usuários comuns é identificado pelo símbolo **\$**

```
aluno@debian:/tmp$
```

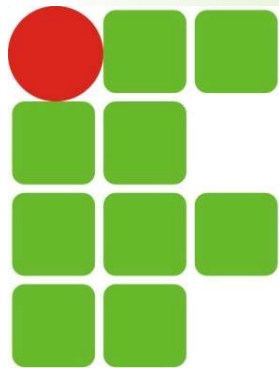
```
^      ^      ^ ^
```

```
|      |      | |__> Usuário comum
```

```
|      |      |____> Diretório atual
```

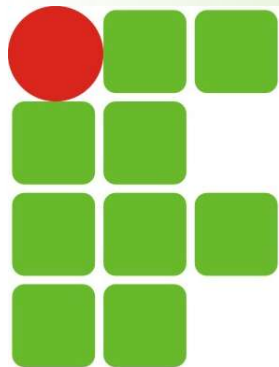
```
|      |____> Nome da máquina
```

```
|____> Nome do usuário
```



Interpretador de comandos

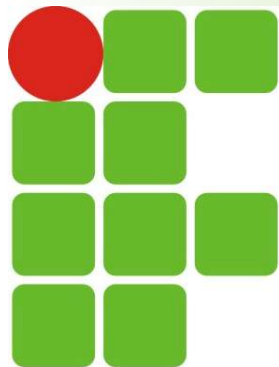
- Os comandos podem ser enviados ao Shell de duas maneiras: interativa e não-interativa
- Interativa
 - Os comandos são digitados no aviso de comando (*Prompt*) e passados ao interpretador de comandos um a um. Neste modo, o computador depende do usuário para executar uma tarefa, ou próximo comando



Interpretador de comandos

■ Não-interativa

- São usados arquivos de comandos criados pelo usuário (scripts) para o computador executar os comandos na ordem encontrada no arquivo. Neste modo, o computador executa os comandos do arquivo um por um e dependendo do término do comando, o script pode checar qual será o próximo comando que será executado e dar continuidade ao processamento.
- Este sistema é útil quando temos que digitar por várias vezes seguidas um mesmo comando ou para compilar algum programa complexo



Interpretador de comandos

■ Estrutura da linha de comando

- A maioria dos comandos na linha de comando segue a mesma estrutura básica:

comando *[opção(ões) /parâmetro(s) ...]* *[argumento(s) ...]*

- Exemplo: `ls -l /home`

- Finalidade de cada componente:

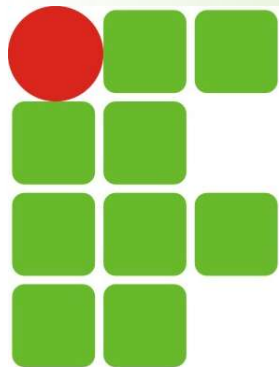
- **Comando:** programa que o usuário pretende executar (`ls`)
- **Opção(ões)/Parâmetro(s):** um "botão" que modifica o comportamento do comando de alguma forma, (`-l`). Podem ser acessadas na forma curta ou longa (`--format = long`)

OBS: É possível combinar múltiplas opções e, no caso do formato abreviado, as letras geralmente podem ser digitadas juntas. Ex:

■ `ls -al`

■ `ls -a -l`

■ `ls --all --format=long`



Interpretador de comandos

■ Estrutura da linha de comando

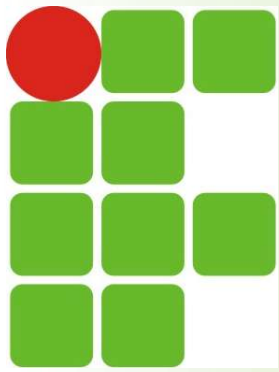
comando *[opção (ões) /parâmetro (s) ...]* *[argumento (s) ...]*

■ Ex: `ls -l /home`

■ Finalidade de cada componente (cont.):

- Argumento(s): dados adicionais exigidos pelo programa, como um nome de arquivo ou caminho (`/home`)
- A única parte **obrigatória** dessa estrutura é o próprio **comando**. Em geral, todos os outros elementos são opcionais, mas um programa pode exigir que determinadas opções, parâmetros ou argumentos sejam especificados
- **OBS:** A maioria dos comandos exibe uma visão geral do próprio comando quando executado com o parâmetro `--help`

■ Ex: `ls --help`



Interpretador de comandos

■ Tipos de comportamento dos comandos

■ O `shell` aceita dois tipos de comandos:

■ Internos

- Comandos que fazem parte do próprio shell e não são programas separados. Existem cerca de 30 desses comandos. Seu principal objetivo é executar tarefas dentro do shell. Ex: `cd`, `set`, `export`

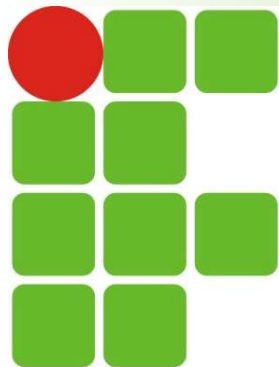
■ Externos

- Comandos que residem em arquivos individuais, e geralmente são programas binários ou scripts. Ao serem executados, o `shell` usa a variável `PATH` para procurar o comando. Além dos programas instalados com o gerenciador de pacotes da distribuição, os usuários também podem criar seus próprios comandos externos

■ O comando **type** mostra a que tipo pertence um comando. Exs:

```
$ type echo
echo is a shell builtin
```

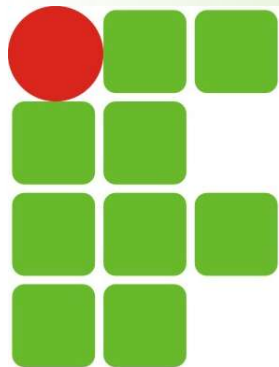
```
$ type man
man is /usr/bin/man
```



Interpretador de comandos

■ Citação

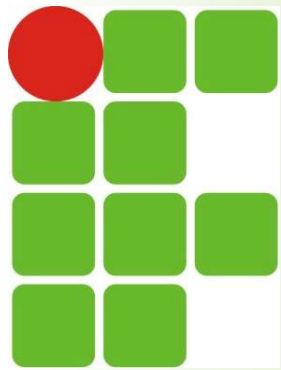
- Como usuário de Linux, você precisa saber criar ou manipular arquivos ou variáveis de diversas maneiras
- Isso é fácil quando se trabalha com nomes de arquivos curtos e valores simples, mas fica mais complicado quando, por exemplo, precisamos usar espaços, caracteres especiais e variáveis
- Os `shells` oferecem um recurso chamado citação, que encapsula esses dados usando diversos tipos de aspas (`"`, `'`). No `bash`, há três tipos de citações:
 - Aspas duplas
 - Aspas simples
 - Caracteres de escape
- Assim, os comandos a seguir não agem da mesma maneira devido à citação:



Interpretador de comandos

■ Citação

```
$ TWOWORDS="two words"
$ touch $TWOWORDS
$ ls -l
-rw-r--r-- 1 carol carol      0 Mar 10 14:56 two
-rw-r--r-- 1 carol carol      0 Mar 10 14:56 words
$ touch "$TWOWORDS"
$ ls -l
-rw-r--r-- 1 carol carol      0 Mar 10 14:56  two
-rw-r--r-- 1 carol carol      0 Mar 10 14:58 'two words'
-rw-r--r-- 1 carol carol      0 Mar 10 14:56  words
$ touch '$TWOWORDS'
$ ls -l
-rw-r--r-- 1 carol carol      0 Mar 10 15:00 '$TWOWORDS'
-rw-r--r-- 1 carol carol      0 Mar 10 14:56  two
-rw-r--r-- 1 carol carol      0 Mar 10 14:58 'two words'
-rw-r--r-- 1 carol carol      0 Mar 10 14:56  words
```



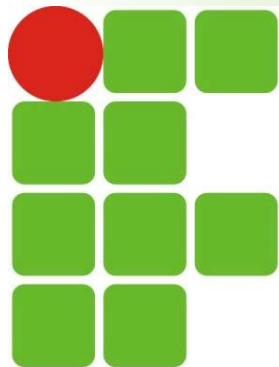
Interpretador de comandos

■ Citação

■ Aspas duplas

- As aspas duplas dizem ao `shell` para considerar o texto entre as aspas ("...") como caracteres regulares
- Todos os caracteres especiais perdem o significado, exceto `$` (cifrão), `\` (barra invertida) e ``` (crase)
- Isso significa que as variáveis, substituições de comando e funções aritméticas ainda podem ser usadas
- Por exemplo, a substituição da variável `$USER` não é afetada pelas aspas duplas:

```
$ echo I am $USER
I am tom
$ echo "I am $USER"
I am tom
```



Interpretador de comandos

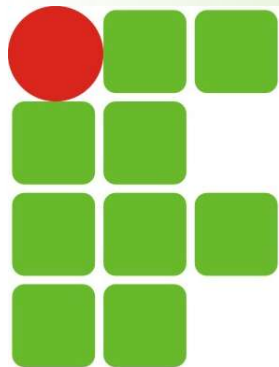
- Citação

- Aspas duplas

- Um caractere de espaço, por outro lado, perde seu significado como separador de argumentos:

```
$ touch new file
$ ls -l
-rw-rw-r-- 1 tom students 0 Oct 8 15:18 file
-rw-rw-r-- 1 tom students 0 Oct 8 15:18 new
$ touch "new file"
$ ls -l
-rw-rw-r-- 1 tom students 0 Oct 8 15:19 new file
```

No 1º exemplo o comando `touch` interpreta as duas seqüências como argumentos individuais e cria dois arquivos separados. No 2º exemplo o comando interpreta as duas seqüências como um único argumento e, portanto, cria apenas um arquivo. No entanto, recomenda-se evitar o caractere de espaço em nomes de arquivos. Em vez disso, é preferível usar um sublinhado (`_`) ou um ponto (`.`).



Interpretador de comandos

■ Citação

■ Aspas simples

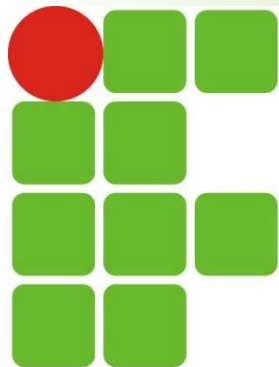
- As aspas simples não têm as exceções das aspas duplas. Elas revogam qualquer significado especial de cada caractere. Se retomarmos um dos primeiros exemplos anteriores:

```
$ echo I am $USER  
I am tom
```

- Ao aplicar aspas simples, obtemos um resultado diferente:

```
$ echo 'I am $USER'  
I am $USER
```

- O comando agora exibe a sequência exata, sem substituir a variável



Interpretador de comandos

■ Citação

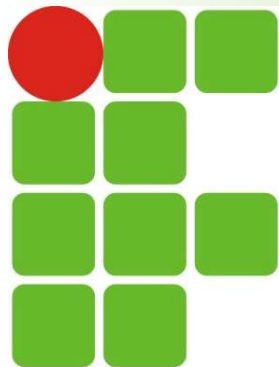
■ Caracteres de escape

- Podemos usar caracteres de escape para remover os significados especiais dos caracteres do `bash` (ex: var. ambiente `$USER`). Por padrão, seu conteúdo é exibido no terminal. Porém, se colocarmos o caractere de barra invertida (`\`) antes do `$`, seu significado especial é cancelado, não permitindo que o `bash` expanda o valor da variável para o nome de usuário que está executando o comando, mas interpretando o nome da variável literalmente

```
$ echo $USER  
carol
```

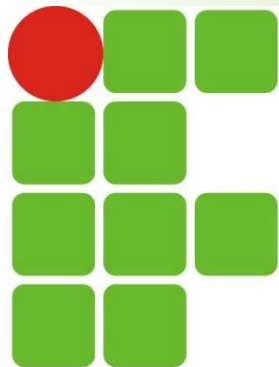
```
$ echo \$USER  
$USER
```

- Apesar de ser possível obter resultados semelhantes usando aspas simples, o funcionamento do caractere de escape é diferente, pois instrui o `bash` a ignorar qualquer significado especial do caractere que ele precede



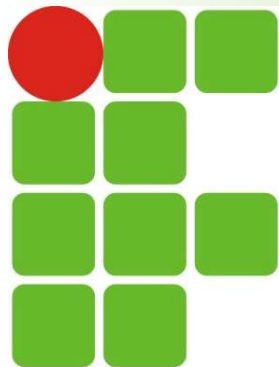
Interpretador de comandos

- Todos os `shells` gerenciam um conjunto de informações de status ao longo das sessões
 - Essas informações de tempo de execução podem mudar durante a sessão e influenciar o comportamento do `shell`, e também são usados pelos programas para determinar aspectos da configuração do sistema
 - A maioria desses dados é armazenada nas chamadas **variáveis** (espaços de armazenamento para dados, como texto ou números). Uma vez definida, seu valor pode ser acessado posteriormente, através de seu nome
 - Seu conteúdo pode ser alterado
 - Recurso muito comum na maioria das linguagens de programação



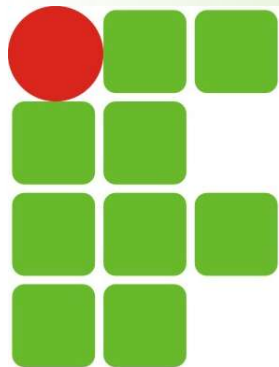
Interpretador de comandos

- A maioria dos `shells` do Linux, existem dois tipos de variáveis:
 - Variáveis locais: disponíveis apenas para o processo atual do `shell`. Se você criar uma variável local e, em seguida, iniciar outro programa nesse `shell`, a variável não poderá mais ser acessada por esse programa
 - Como **não** são herdadas por subprocessos, essas variáveis são chamadas variáveis locais.



Interpretador de comandos

- A maioria dos `shells` do Linux, existem dois tipos de variáveis:
 - Variáveis de ambiente: disponíveis tanto em uma sessão de `shell` específica quanto em subprocessos gerados a partir dessa sessão de `shell`
 - Podem ser usadas para transmitir dados de configuração para os comandos executados. Como os programas podem acessar essas variáveis, elas são chamadas **variáveis de ambiente**
 - A maioria destas variáveis aparece em letras maiúsculas (ex: `PATH`, `DATE`, `USER`).
 - As variáveis de ambiente padrão fornece, p ex, informações sobre o diretório inicial ou o tipo de terminal do usuário, etc.
 - Variáveis **não** são persistentes. Quando o shell em que foram configuradas é fechado, todas as variáveis e seus conteúdos são perdidos, pois são armazenadas em arquivos de configuração



Interpretador de comandos

■ Manipulação de variáveis

- Como administrador do sistema, você precisará criar, modificar ou remover variáveis locais e de ambiente
- Você pode configurar uma **variável local** usando o operador = (igual), através de uma atribuição simples:

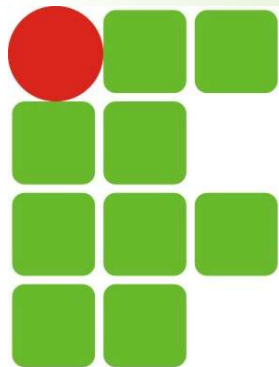
```
$ greeting=hello
```

- **OBS: Não** coloque espaço antes ou depois do operador =
- É possível exibir qualquer variável usando o comando `echo`. O comando geralmente exibe o texto na seção de argumentos:

```
$ echo greeting  
greeting
```

```
$ echo $greeting  
hello
```

- Abra outro `shell` e tente exibir o conteúdo da variável



Interpretador de comandos

■ Manipulação de variáveis

- Podemos também atribuir a uma variável o resultado de um comando (além de parâmetros e argumentos, caso existam). Pode ser feito de duas formas:

- Colocando o comando entre crase (`)

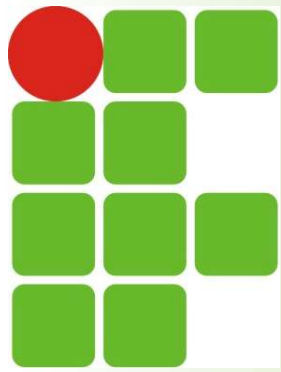
```
variavel=`comando`
```

- Colocando o comando entre parênteses, precedido do cifrão (\$)

```
variavel=$(comando)
```

- Exemplos:

```
isa@isa20241:~$ processo=`ps`  
isa@isa20241:~$ echo $processo  
PID TTY TIME CMD 923 tty1 00:00:00 bash 1027 tty1 00:00:00 ps  
isa@isa20241:~$ processo2=$(ps)  
isa@isa20241:~$ echo $processo2  
PID TTY TIME CMD 923 tty1 00:00:00 bash 1028 tty1 00:00:00 ps
```

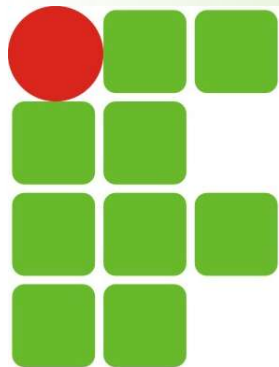


Interpretador de comandos

- Para verificar se a variável é local, gere um novo processo e veja se este acessa a variável
 - Inicie outro `shell` e execute nele o comando `echo`
 - Como o novo shell roda em um novo processo, ele não herdará as variáveis locais do processo pai:
- Para remover uma variável, usamos o comando `unset` e passando o **nome da variável** como argumento. Ex:

```
$ echo $greeting world
hello world
$ bash -c 'echo $greeting world'
world
```

```
$ unset greeting
```

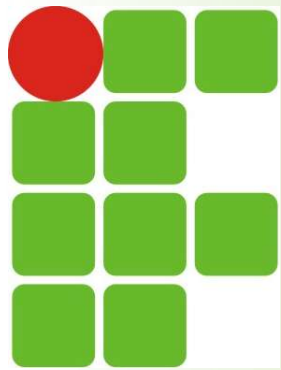


Interpretador de comandos

- Trabalhando com variáveis globais
 - Para disponibilizar uma variável local para subprocessos, podemos transformá-la em variável de ambiente. Usamos para isso o comando `export`. Quando ele é invocado junto ao **nome da variável**, esta é adicionada ao ambiente do shell:
- Outra forma de criar a variável de ambiente é combinar os dois comandos acima, atribuindo o valor da variável na parte de argumento do comando

```
$ greeting=hello  
$ export greeting
```

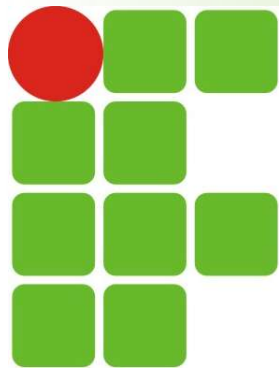
```
$ export greeting=hey
```



Interpretador de comandos

- Trabalhando com variáveis globais
 - Vamos conferir novamente se a variável está acessível aos subprocessos:

```
$ export greeting=hey  
$ echo $greeting world  
hey world  
$ bash -c 'echo $greeting world'  
hey world
```

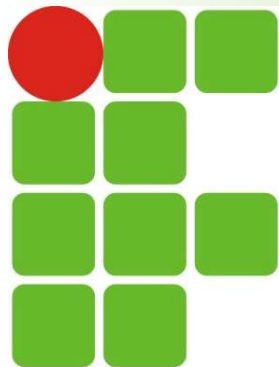



Interpretador de comandos

- Trabalhando com variáveis globais
 - Outra forma de usar variáveis de ambiente é colocá-las na frente dos comandos. Ex: a variável de ambiente TZ contém o fuso horário. Essa variável é usada pelo comando `date` para determinar qual hora do fuso horário deve ser exibida:

```
$ TZ=EST date
Thu 31 Jan 10:07:35 EST 2019
$ TZ=GMT date
Thu 31 Jan 15:07:35 GMT 2019
```

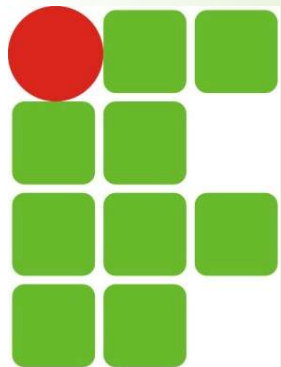
- **OBS:** O comando `env` exibe todas as variáveis de ambiente



Interpretador de comandos

- Trabalhando com variáveis globais
 - A variável `PATH` é uma das variáveis de ambiente mais importantes, pois armazena uma lista de diretórios (separados por `:`) que contêm programas executáveis que funcionam como comandos do shell do Linux
 - Para acrescentar um novo diretório à variável, usamos o sinal de dois pontos (`:`): `PATH=$PATH:new_directory`
 - Exemplo:

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
$ PATH=$PATH:/home/user/bin
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/home/user/bin
```



Interpretador de comandos

- Trabalhando com variáveis globais
 - Para descobrir como o `shell` chama um comando específico, `which` pode ser executado com o ***nome do comando como argumento***. Ex:

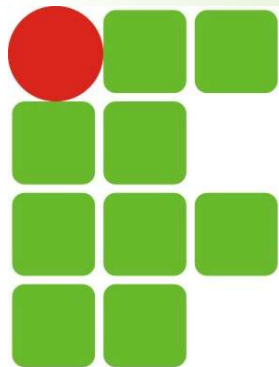
```
$ which nano
```

```
/usr/bin/nano
```

 - Neste exemplo, o executável `nano` está localizado no diretório `/usr/bin`. Vamos remover o diretório da variável e verificar se o comando ainda funciona:

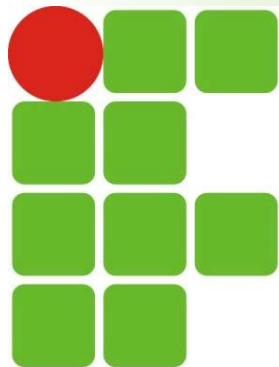
```
$ PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/sbin:/bin:/usr/games
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/sbin:/bin:/usr/games
$ which nano
which: no nano in (/usr/local/sbin:/usr/local/bin:/usr/sbin:/sbin:/bin:/usr/games)
```

OBS: a ordem dos diretórios em `PATH` é a ordem de pesquisa



Interpretador de comandos

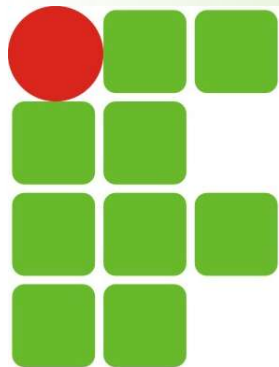
- Usando vários terminais
 - Podemos utilizar vários terminais simultaneamente
 - No modo texto (CLI) basta pressionar as teclas **Alt + F1**, **Alt + F2**, ..., **Alt + F6**
 - No modo gráfico basta abrir tantos quantos forem necessários
 - Ex: clicando em Aplicativos → Acessórios → ***Terminal***
 - A partir de um terminal aberto, pode se usar o menu "Arquivo", ou usar as teclas "Shift + Ctrl + n"
 - Caso o ambiente gráfico já esteja aberto, podemos pressionar **Ctrl + Alt + F3** para ir para o modo texto
 - A tecla F3 pode variar, sendo F1 em distribuições mais antigas
 - Para voltar ao ambiente gráfico basta pressionar **Alt + F7**



Interpretador de comandos

- O sistema de arquivos do Linux é semelhante ao de outros SOs, pois contém **arquivos** e **diretórios**
 - **Arquivos** contêm dados, como texto legível para humanos, programas executáveis ou dados binários usados pelo computador.
 - **Diretórios** são usados para organizar o sistema de arquivos. Eles podem conter arquivos e outros diretórios.

```
$ tree
Documents
├── Mission-Statement.txt
└── Reports
    └── report2018.txt
1 directory, 2 files
```

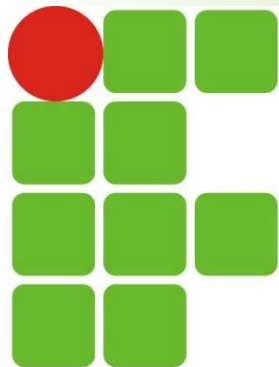


Interpretador de comandos

■ Arquivos e diretórios

- Nomes de arquivos e diretórios no Linux podem conter letras minúsculas e maiúsculas, números, espaços e caracteres especiais
- Porém, como muitos caracteres especiais têm um significado preciso no shell do Linux, é recomendável não usar espaços ou caracteres especiais ao nomear arquivos ou diretórios
- Para usar espaços, por exemplo, é preciso que o caractere de escape \ seja inserido corretamente:

```
cd Ronaldo\ Maia
```

Interpretador de comandos

- Navegando no sistema de arquivos

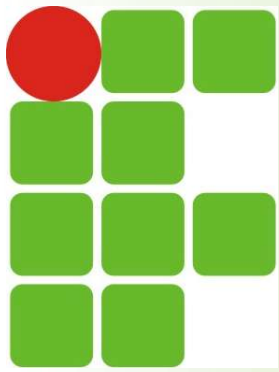
- Os `shells` são baseados em texto, e por isso é importante conhecer sua localização atual ao navegar no sistema de arquivos. O prompt de comando fornece essas informações:

```
user@hostname ~/Documentos/IFRN $
```

- Graças ao prompt, sabemos que nossa localização atual é o diretório IFRN. O comando `pwd` serve para imprimir o diretório de trabalho:

```
user@hostname ~/Documentos/IFRN $ pwd  
/home/user/Documentos/IFRN
```

- Perceba que saída do comando `pwd` difere um pouco do caminho mostrado no prompt de comando. Em vez de `/ home/IFRN`, este contém um til (`~`), caractere especial que representa o diretório do usuário
- A relação entre os diretórios é representada com uma barra (`/`). **IFRN** é um subdiretório de **Documentos**, que por sua vez é um subdiretório de **user**, localizado em um diretório **home**
 - O **home** não parece ter um diretório pai, mas isso não é verdade. O pai de **home** se chama **root** e é representado pela primeira barra (`/`)



Interpretador de comandos

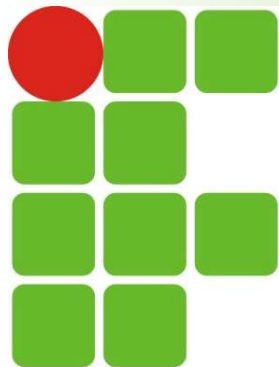
- Navegando no sistema de arquivos

- O conteúdo do diretório atual é listado com o comando `ls`

```
user@hostname ~/Documents/Reports $ ls  
report2018.txt
```

- Note que o `ls` não fornece informações sobre o diretório pai. Da mesma forma, por padrão, `ls` não exibe nenhuma informação sobre o conteúdo dos subdiretórios, mostrando apenas o que está no diretório atual
- A navegação no Linux é feita principalmente com o comando `cd`, que muda o diretório atual

```
user@hostname ~ $ cd /home/user/Documents  
user@hostname ~/Documents $ ls  
Mission-Statement.txt Reports  
user@hostname ~/Documents $ cd Reports  
user@hostname ~/Documents/Reports $
```



Interpretador de comandos

- Navegando no sistema de arquivos

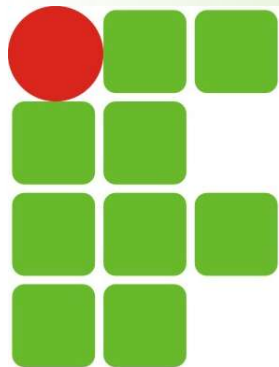
- Caminhos absolutos e relativos

- O comando `pwd` sempre imprime um **caminho absoluto**, ou seja, o caminho que contém todas as etapas, desde o início do sistema de arquivos (/) até o diretório desejado. Exemplo:

```
user@hostname ~ $ cd /home/user/Documents/IFRN
```

- O caminho absoluto contém todas as informações necessárias para se chegar ao diretório pretendido a partir de qualquer lugar no sistema de arquivos. Desvantagem: é "tedioso" digitá-lo inteiro
 - O **caminho relativo** é mais fácil de digitar, pois são mais curtos. Porém, só tem significado em relação à sua localização atual. Exemplo:

```
user@hostname ~ $ cd Documentos/IFRN
```



Interpretador de comandos

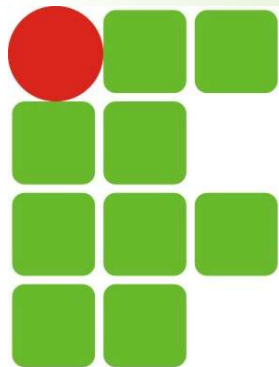
- Navegando no sistema de arquivos

- Caminhos relativos especiais

- O `shell` do Linux oferece maneiras de encurtar os caminhos durante a navegação. Para revelar os primeiros caminhos especiais, inserimos o comando `ls` com a opção `-a`, que modifica o comando `ls` para que todos os arquivos e diretórios sejam listados, incluindo os arquivos e diretórios ocultos. Exemplo:

```
user@hostname ~/Documents/Reports $ ls -a
.
..
report2018.txt
```

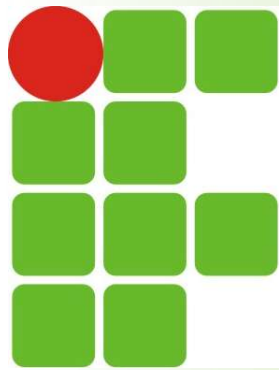
- Esse comando revelou dois resultados adicionais que não representam novos arquivos ou diretórios, mas sim diretórios conhecidos:
 - `"."`: indica o **diretório atual** ou **dir. de trabalho** (Reports)
 - `".."`: indica o **diretório pai** ou **dir. superior** (Documents)



Interpretador de comandos

- Navegando no sistema de arquivos
 - Caminhos relativos especiais
 - Geralmente não é necessário usar o caminho relativo especial para o local atual. É mais simples digitar `report2018.txt` que `./Report2018.txt`
 - Exemplo de caminho relativo para subir na árvore de diretórios:

```
user@hostname ~/Documents/Reports $ cd ..
user@hostname ~/Documents $ cd ../..
```
 - O **diretório anterior** é identificado por um traço `"-"`
 - É útil para retornar ao último diretório usado
 - Se estiver no diretório **/usr/local** e digitar `cd /lib`, você pode retornar facilmente para **/usr/local** usando `cd -`



Referências

- LPI Linux Essentials
 - Tópico 2: Encontrando seu caminho em um Sistema Linux (Seções 2.1 e 2.3)
<https://learning.lpi.org/pt/learning-materials/010-160/?authuser=0>
- Material de aula do prof. Carlos Gustavo
- Material de aula do prof. Francisco Sales